

P1372R0

Giving `atomic_ref` implementers more flexibility by providing customization points for non-lock-free implementation

Published Proposal, 2018-11-25

This version:

wg21.link/P1372

Issue Tracking:

[GitHub](#)

Author:

[David S. Hollman](#)

Audience:

SG1, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Revision History

1.1 P1372R0

2 Motivation

3 Design

4 Wording

References

Informative References

§ 1. Revision History

§ 1.1. P1372R0

- Initial version

§ 2. Motivation

[P1298R0], presented at the 2018-11 San Diego meeting, revealed by way of motivation a number of difficulties surrounding the implementation of `atomic_ref`. Though the author withdrew the paper, its presence has served as a catalyst for further discussion. The basic issue surrounded the implementation of `atomic_ref` for types that cannot be supported in a lock-free manner on certain platforms. In the absence of transactional memory, the current proposal essentially mandates that `atomic_ref` for these types must use an address-based, sharded lock table. For many implementations dealing with issues on the fringe of standard C++ (such as dynamic loading of shared libraries and heterogeneity), implementors have expressed the concern that this approach is impractical or impossible. While this paper does not provide a fix for this in the absolutely most general case, it provides a portable means for users to opt in to an alternative implementation that could be significantly more efficient or practical in those cases.

§ 3. Design

We propose a pair of customization point objects, named `lock_reference` and `unlock_reference`, that can be customized for types `T` where `atomic_ref<T>::is_always_lock_free` is both `true` and `false`. The customization point object mechanism used here is exactly analogous to those for ranges (and thus needs no further discussion here). There are a couple of design questions here, though:

- Should this customization point be used only when `atomic_ref<T>::is_always_lock_free` is `false`, or should the customization also be used in the `true` case?
 - *Argument in favor*: if the user has a reason (e.g., debugging) for customizing the case where lock-free atomics are available, perhaps the implementation should honor this desire.
 - *Argument against*: in portable code, the user may wish to provide the customizations because some target implementations will support lock-free atomics on the given type and some will not.

- **Decision:** given that one of the design goals `atomic_ref<T>` in the first place is portable use of atomic semantics across lock-free and non-lock-free implementations, the design of the customization point should mirror this design intent and allow customization of `T` to be ignored when a lock-free implementation is available.
- Should the implementation be *required* to use the customization point when `atomic_ref<T>::is_always_lock_free` is `false`?
 - *Argument in favor:* Consistency, simplicity, predictable behavior.
 - *Argument against:* Some implementations may have access to strategies that, while not technically lock-free, may be significantly more efficient than a user can provide in an implementation-independent context (such as OS-emulated transactional memory).
 - **Decision:** Make it optional for now, but change paper to reflect results of straw poll along these lines.
- Should freestanding implementations be allowed to omit `atomic_ref<T>` for types that do not provide a customization of this property?
 - *Argument in favor:* In general, creation of an efficient sharded lock table may be impractical or impossible in freestanding implementations, particularly when those implementations need to inter-operate with a variety of full implementations.
 - **Decision:** Address this in a follow-up paper to limit the scope of the current paper.
- Should the memory order argument be propagated to the customization points?
 - *Argument against:* It is unclear how any implementation of `lock_reference` or `unlock_reference` would use this information, and it can always be added later.
 - **Decision:** No.

§ 4. Wording

Make the following changes to `[atomics.ref.generic]/3`:

The lifetime (**[basic.life]**) of an object referenced by `*ptr` shall exceed the lifetime of all `atomic_refs` that reference the object. While any `atomic_ref` instances exist that reference the `*ptr` object, all accesses to that object shall exclusively occur through those `atomic_ref` instances or through objects derived from the parameters to customization point objects `lock_reference` and `unlock_reference` in `[atomics.ref.customization]` during the execution of the customization. No subobject of the object referenced by `atomic_ref` shall be concurrently referenced by any other `atomic_ref` object.

Make the following changes to the note in `[atomics.ref.generic]/4`:

[Note: Atomic operations or the `atomic_ref` constructor could acquire a shared resource, such as a lock associated with the referenced object, to enable atomic operations to be applied to the referenced object. This may (but not must) take the form of a pair of calls to the `lock_reference` and `unlock_reference` customization points on `T` where `atomic_ref<T>::is_always_lock_free` is false. —end note]

Add the following paragraph after `[atomics.ref.operations]/3`:

[Note: The value of `required_alignment` may depend on the presence or absense of the `lock_reference` and `unlock_reference` customization points if `is_always_lock_free` is false. —end note]

Add the following section to the end of `[atomics.ref.generic]`:

29.6. ♦ Customization points

The following customization points affect the behavior of `atomic_ref<T>` for any `T` such that `atomic_ref<T>::is_always_lock_free` is false. An implementation must not instantiate these customization points for any `T` such that `atomic_ref<T>::is_always_lock_free` is true.

Signature	Semantics
<pre>template <class T> void lock_reference(T& obj);</pre>	Customization: Dispatches to <code>obj.lock_reference()</code> if that expression is well-formed; otherwise, dispatches to (unqualified) <code>lock_reference(obj)</code> in a context that doesn't include the <code>std::lock_reference</code> customization point object.

Semantics:

Obtains exclusive access to `obj` until `std::unlock_reference` is called on an object with the same address as `obj`. This customization shall not return while another thread of execution has locked an object with the same address and type but has not unlocked that reference, which shall have a *happens-before* relationship with this customization point.

```
template <class T>  
void unlock_reference(T& obj);
```

Customization:

Dispatches to `obj.unlock_reference()` if that expression is well-formed; otherwise, dispatches to (unqualified) `unlock_reference(obj)` in a context that doesn't include the `std::unlock_reference` customization point object.

Semantics:

Relinquishes exclusive access to `obj` that was previously locked on the same thread of execution with `std::lock_reference` on an instance of `obj` having the same type and address is called on an object with the same address and type of `obj`.
[Note: An implementation of `atomic_ref` must ensure calls to these customization points match in the described manner. —end note]

§ References

§ Informative References

[P1298R0]

Olivier Giroux. [Reserve more freedom for atomic_ref<> implementers](https://wg21.link/p1298r0). 8 October 2018. URL: <https://wg21.link/p1298r0>